Task1

```java
class Animal {  2 usages  1 inheritor
    void sound() {  1 usage  1 override
        System.out.println("Sounds of different animals");
    }
}

class Cat extends Animal {  1 usage
    @Override  1 usage
    void sound() {
        System.out.println("Meow is the sound of cat");
    }
}
```

Rename usages

```java
public class Sound {
    public static void main(String[] args) {
        Animal obj = new Cat();
        obj.sound(); // Meow is the sound of cat
    }
}
```

```
"C:\Program Files\Java\jdk-17\bin\java.
Meow is the sound of cat

Process finished with exit code 0
```

Task2

```java
class Cat extends Animal {  3 usages
        System.out.println(" Meow is the sound of cat");
    }
}

class Task2 {
    // Utility method to print any list
    static void printList(List<?> list) {  1 usage
        for (Object element : list) {
            System.out.println(element);
        }
    }

    public static void main(String[] args) {
        Animal obj = new Cat();
        obj.sound(); // Meow is the sound of cat

        List<Cat> clist = new ArrayList<>();
        clist.add(new Cat());

        printList(clist);
    }
}
```

```
 Meow is the sound of cat
Cat@eed1f14

Process finished with exit code 0
```

Task3

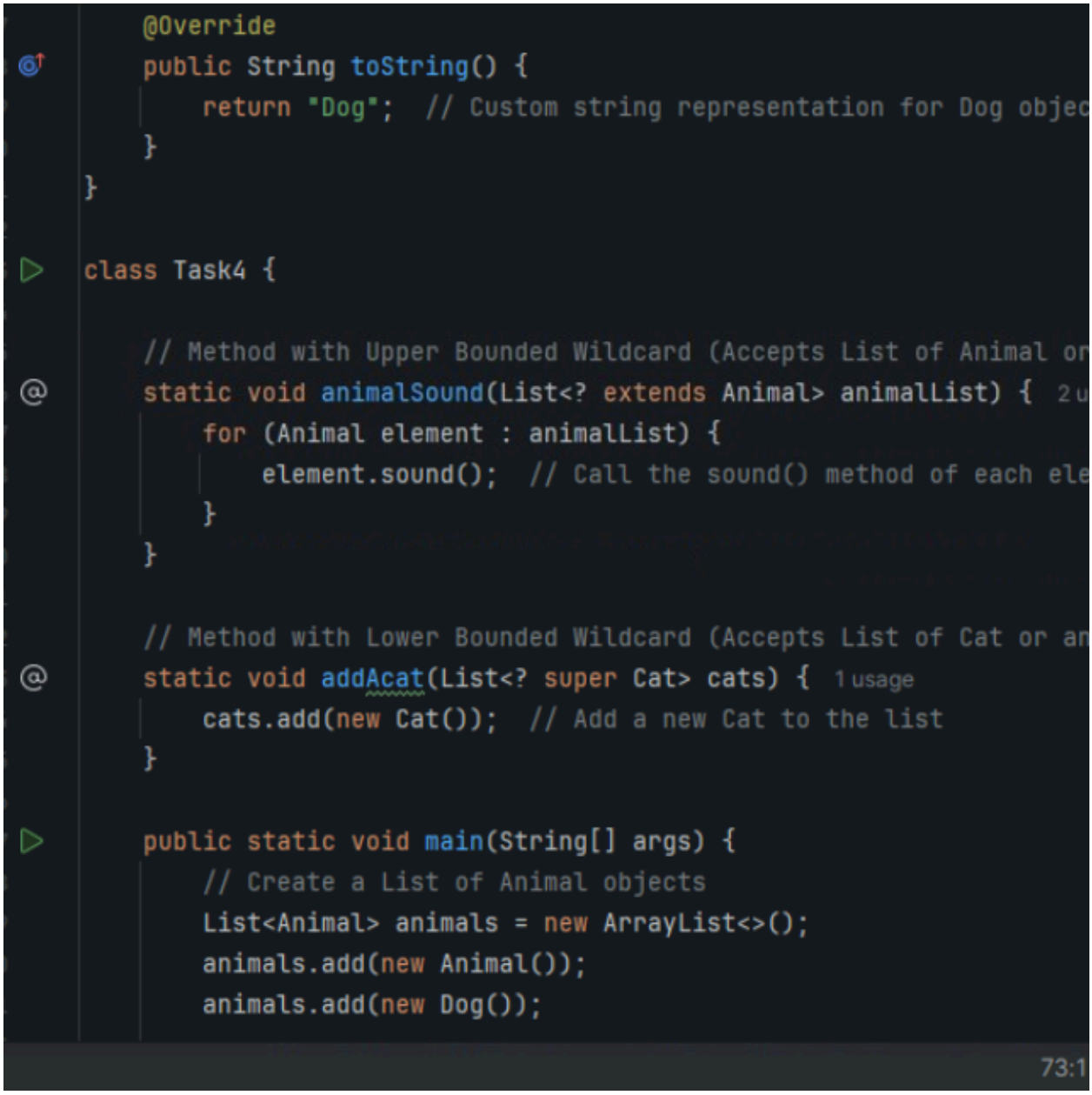
```java
import java.util.*;

class Animal {  4 usages  2 inheritors
    void sound() {  1 usage  2 overrides
        System.out.println("Sounds of different animals");
    }
}

class Cat extends Animal {  2 usages
    @Override  1 usage
    void sound() {
        System.out.println("Meow is the sound of the cat");
    }
}

class Dog extends Animal {  2 usages
    @Override  1 usage
    void sound() {
        System.out.println("Woof is the sound of the dog");
    }
}

class Task3 {

    // Upper Bounded Wildcard Method that works for any subclass of Animal
    static void animalSound(List<? extends Animal> animalList) {  2 usages
        for (Animal element : animalList) {
```

```java
class Task3 {

    // Upper Bounded Wildcard Method that works for any subclass of Animal
    static void animalSound(List<? extends Animal> animalList) {  2 usages
        for (Animal element : animalList) {
            element.sound();  // Calls the sound() method of Animal (or su
        }
    }

    public static void main(String[] args) {
        // Create a List of Cats
        List<Cat> cats = new ArrayList<>();
        cats.add(new Cat());

        // Create a List of Dogs
        List<Dog> dogs = new ArrayList<>();
        dogs.add(new Dog());

        // Call animalSound() with different types of animals
        System.out.println("Cat sounds:");
        animalSound(cats);  // Calls the sound() method of Cat

        System.out.println("\nDog sounds:");
        animalSound(dogs);  // Calls the sound() method of Dog
    }
}
```

```
Meow is the sound of the cat

Dog sounds:
Woof is the sound of the dog

Process finished with exit code 0
```

Task4

```java
import java.util.*;

class Animal {  6 usages  2 inheritors
    void sound() {  1 usage  2 overrides
        System.out.println("Sounds of different animals");
    }
}

class Cat extends Animal {  4 usages
    @Override  1 usage
    void sound() {
        System.out.println("Meow is the sound of the cat");
    }

    @Override
    public String toString() {
        return "Cat";  // Custom string representation for Cat objec
    }
}

class Dog extends Animal {  1 usage
    @Override  1 usage
    void sound() {
        System.out.println("Woof is the sound of the dog");
    }

    @Override
```



```
    @Override
    public String toString() {
        return "Dog";  // Custom string representation for Dog objec
    }
}

class Task4 {

    // Method with Upper Bounded Wildcard (Accepts List of Animal or
    static void animalSound(List<? extends Animal> animalList) {  2 u
        for (Animal element : animalList) {
            element.sound();  // Call the sound() method of each ele
        }
    }

    // Method with Lower Bounded Wildcard (Accepts List of Cat or an
    static void addAcat(List<? super Cat> cats) {  1 usage
        cats.add(new Cat());  // Add a new Cat to the list
    }

    public static void main(String[] args) {
        // Create a List of Animal objects
        List<Animal> animals = new ArrayList<>();
        animals.add(new Animal());
        animals.add(new Dog());
```

73:1

```java
        // Create a List of Animal objects
        List<Animal> animals = new ArrayList<>();
        animals.add(new Animal());
        animals.add(new Dog());

        // Call animalSound() with a list of different animals
        System.out.println("Animal sounds:");
        animalSound(animals);  // Calls the sound() method of Animal and Dog

        // Create a List of Cat objects
        List<Cat> cats = new ArrayList<>();
        cats.add(new Cat());

        // Call animalSound() with a list of Cats
        System.out.println("\nCat sounds:");
        animalSound(cats);  // Calls the sound() method of Cat

        // Add a Cat to a List of Animals using Lower Bounded Wildcard
        System.out.println("\nAdding a Cat to a list of Animals:");
        addAcat(animals);  // This will add a Cat to the animals list

        // Print the animals list to verify that the Cat has been added
        System.out.println(animals);  // Should output: [Animal, Dog, Cat]
    }
```

```
"C:\Program Files\Java\jdk-17\
Animal sounds:
Sounds of different animals
Woof is the sound of the dog

Cat sounds:
Meow is the sound of the cat
```

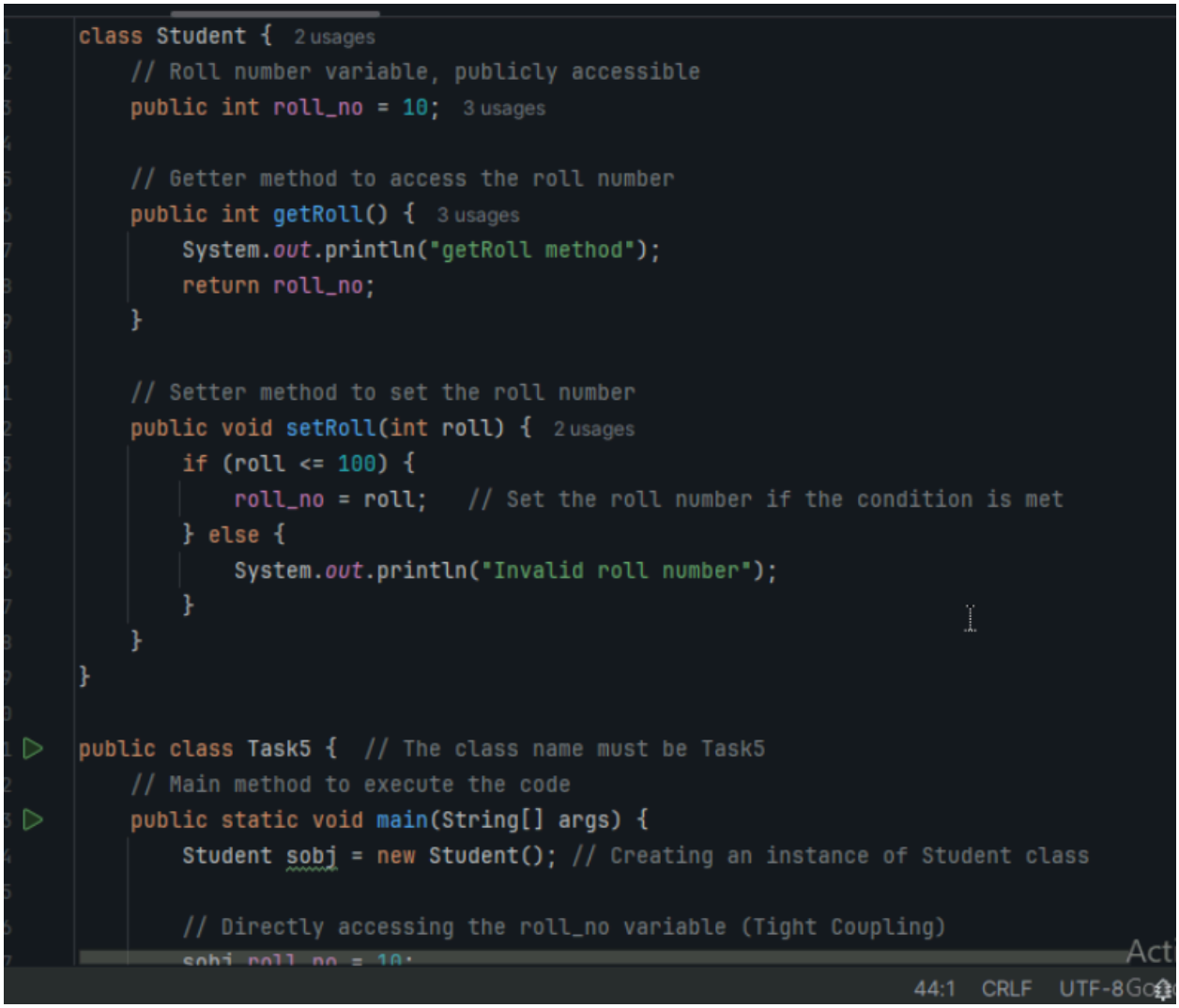
```
Cat sounds:
Meow is the sound of the cat

Adding a Cat to a list of Animals:
[Animal@7229724f, Dog, Cat]
```

Task 5

```java
public class DIP {
    public static void main(String[] args) {
        lightSwitch.operate();  // Output: Light turned on

        // If we were to add another device like Fan:
        SwitchOnOff fan = new Fan();
        Switch fanSwitch = new Switch(fan);
        fanSwitch.operate();  // Output: Fan is running
    }
}
```

```
"C:\Program Files\Java\jdk-17\bin
Light turned on
Fan is running

Process finished with exit code 0
```

tASK 5
tight coupling



```java
class Student {  2 usages
    // Roll number variable, publicly accessible
    public int roll_no = 10;  3 usages

    // Getter method to access the roll number
    public int getRoll() {  3 usages
        System.out.println("getRoll method");
        return roll_no;
    }

    // Setter method to set the roll number
    public void setRoll(int roll) {  2 usages
        if (roll <= 100) {
            roll_no = roll;   // Set the roll number if the condition is met
        } else {
            System.out.println("Invalid roll number");
        }
    }
}

public class Task5 {  // The class name must be Task5
    // Main method to execute the code
    public static void main(String[] args) {
        Student sobj = new Student(); // Creating an instance of Student class

        // Directly accessing the roll_no variable (Tight Coupling)
        sobj.roll_no = 10;
```

```
    // Directly accessing the roll_no variable (Tight Coupling)
    sobj.roll_no = 10;

    // Uncomment below line to test with invalid roll number
    // sobj.roll_no = 110;

    // Output the roll number using the getter
    System.out.println("The roll number of the student is " + sobj.getRoll());  //

    // Using setter to set the roll number to a valid value
    sobj.setRoll(50);
    System.out.println("The updated roll number of the student is " + sobj.getRoll

    // Using setter to set the roll number to an invalid value (greater than 100)
    sobj.setRoll(110);  // This will not update
    System.out.println("The roll number after invalid set is " + sobj.getRoll());
}
```

```
getRoll method
The roll number of the student is 10
getRoll method
The updated roll number of the student is 50
Invalid roll number
getRoll method
```

src

Task 6

Lose coupling

```java
// Class representing a Student
class Student implements Person {  1 usage
    // Private roll number, encapsulated properly
    private int roll_no = 0;  2 usages

    // Getter method for roll number
    public int getRoll() {  1 usage
        System.out.println("getRoll method");
        return roll_no;
    }

    // Setter method for roll number
    public void setRoll(int roll) {  1 usage
        if (roll <= 100) {  // Validating roll number to be <= 100
            roll_no = roll;
        } else {
            System.out.println("Invalid roll number");
        }
    }
}

// Interface to create loose coupling (abstract representation of a person)
interface Person {  2 usages  1 implementation
    void setRoll(int roll);  1 usage  1 implementation
    int getRoll();  1 usage  1 implementation
}
```

```java
// Interface to create loose coupling (abstract representation of a person)
interface Person {  2 usages  1 implementation
    void setRoll(int roll);  1 usage  1 implementation
    int getRoll();  1 usage  1 implementation
}

// Main class demonstrating Loose Coupling
public class LooseCoupling {
    public static void main(String[] args) {
        // Loose coupling: Now Student is treated as a Person
        Person sobj = new Student();  // This could be any subclass of Person in the future

        // Set roll number using setter method
        sobj.setRoll(10);

        // Get roll number using getter method
        System.out.println("The roll number of the student is " + sobj.getRoll());  // Expect
    }
}
```

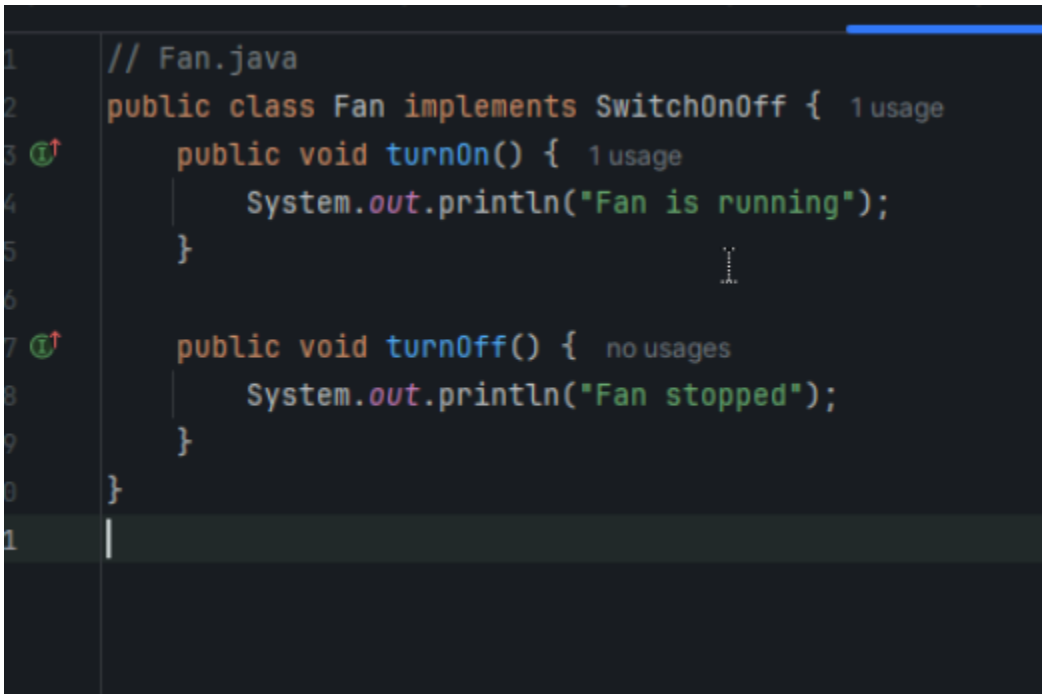
```
"C:\Program Files\Java\jdk-17\bin\java.exe"
getRoll method
The roll number of the student is 10

Process finished with exit code 0
```

Task 7

ain.java SwitchOnOff.java LightBulb.java Fa

```java
// SwitchOnOff.java
interface SwitchOnOff {  no usages  2 implementations
    void turnOn();  no usages  2 implementations
    void turnOff();  no usages  2 implementations
}
```

```java
// LightBulb.java
public class LightBulb implements SwitchOnOff {  no us
    public void turnOn() {  no usages
        System.out.println("Light turned on");
    }

    public void turnOff() {  no usages
        System.out.println("Light is off");
    }
}
```

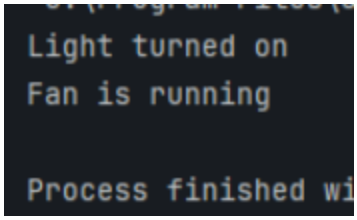


```java
// Fan.java
public class Fan implements SwitchOnOff {  1 usage
    public void turnOn() {  1 usage
        System.out.println("Fan is running");
    }

    public void turnOff() {  no usages
        System.out.println("Fan stopped");
    }
}
```

```java
// Switch.java
public class Switch {  no usages
    private SwitchOnOff device;  2 usages

    // Constructor to initialize the device
    public Switch(SwitchOnOff device) {  no usages
        this.device = device;
    }

    // Method to operate (turn on the device)
    public void operate() {  no usages
        device.turnOn();
    }
}
```

Task8

```java
// DIP.java
public class DIP {
    public static void main(String[] args) {
        // Creating the LightBulb object (could be any device implementing
        SwitchOnOff lightBulb = new LightBulb();

        // Creating a Switch object that can operate the lightbulb
        Switch lightSwitch = new Switch(lightBulb);

        // Operating the light (turning it on)
        lightSwitch.operate();  // Output: Light turned on

        // If we were to add another device like Fan:
        SwitchOnOff fan = new Fan();
        Switch fanSwitch = new Switch(fan);
        fanSwitch.operate();  // Output: Fan is running
    }
}
```



Task 09:

Why should we choose Composition over Inheritance?

Loose coupling

Flexibility

Better encapsulation

1. Loose Coupling

- **Composition** allows objects to be **built from smaller, independent parts**.
- These parts can be replaced or modified **without affecting the rest of the system**.
- Inheritance tightly couples the child class to the parent class, making changes risky.

With composition, you depend on interfaces or behaviors, not fixed hierarchies

Flexibility

- Composition lets you **dynamically change behavior** at runtime by swapping out components.
- Inheritance is **static**—once you inherit, you're stuck with the parent's behavior unless you override it.

You can combine different behaviors from multiple classes without being limited to single inheritance.

Better Encapsulation

- Composition keeps components **self-contained** with clearly defined interfaces.
- Inheritance often **exposes internal details** of the parent to the child class.

Encapsulation leads to better control over how objects interact and prevents unintended dependencies.

Home task

Association

- **Definition**: A **general connection** between two classes.
- It shows that objects of one class **use or interact with** objects of another class.
- It is a **"uses-a"** or **"knows-a"** relationship.

Example:

```
class Student {
        String name;
}

class Course {
        String title;
        void enroll(Student s) {
        // Student is associated with Course
        }
}
```

UML Diagram Notation: A solid line between two classes
Meaning: A `Course` is associated with `Student` — the student enrolls in the course.

Aggregation (a special form of association)

- **Definition**: A **"has-a"** relationship.
- One class **contains** or **owns** another class, but the **contained object can exist independently**.
- Represented in UML with a **hollow diamond**.

Example:

```
class Team {
	List<Player> players; // Team has players
}

class Player {
	String name;
}
```

: A `Team` has `Players`, but a `Player` can exist even if the `Team` is disbanded.

UML Diagram Notation: Line with **white (hollow) diamond** at the "whole" end (Team).

Composition (stronger form of aggregation)

- **Definition**: Also a **"has-a"** relationship, but with **strong ownership**.
- The **lifetime of the contained object depends** on the container.
- Represented in UML with a **filled (black) diamond**.

Example:

```
class House {
	private Room room = new Room(); // Room is part of House
}

class Room {
	String name;
}
```

A `Room` cannot exist independently outside a `House`. If the `House` is destroyed, so are its `Rooms`.

UML Diagram Notation: Line with **black diamond** at the "whole" end (House).

Inheritance (Generalization)

- **Definition**: A **"is-a"** relationship where a subclass **inherits** from a superclass.
- Enables **code reuse** and **polymorphism**.

- Represented in UML with a **hollow triangle arrow** pointing to the superclass.

Example:

```
class Animal {
        void makeSound() {}
}

class Dog extends Animal {
        void makeSound() { System.out.println("Bark"); }
}
```

: A `Dog` **is an** `Animal`, so it inherits properties and behavior from the `Animal` class.

UML Diagram Notation: Solid line with **hollow triangle** pointing to `Animal`.